

Manual Técnico do Script `statistics_dashboard.py` - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL + Bibliotecas de Visualização (Pandas, Matplotlib, Seaborn)

Este manual técnico descreve em detalhes o script `statistics_dashboard.py`, que atua como o motor de geração de estatísticas e dashboards visuais no sistema Hórus. O script é responsável por consultar dados no banco de dados, calcular métricas agregadas (totais, únicos, distribuições) e produzir gráficos estáticos em base64 para integração em interfaces web. Ele foca em insights sobre credenciais vazadas, como totais de registros, senhas/sites/usuários comuns e proporções de duplicatas.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de visualização implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da geração de dashboards.

1. Introdução e Propósito

O `statistics_dashboard.py` é o componente dedicado à criação de dashboards estatísticos no Hórus, uma plataforma de Threat Intelligence para processamento e análise de credenciais vazadas. Ele consulta tabelas como 'credenciais' e 'credenciais_geral' no PostgreSQL, computa métricas (ex: contagens totais, valores únicos, itens comuns) e gera gráficos com Matplotlib/Seaborn para visualização de dados como top sites de vazamentos e proporções de registros únicos/duplicados. O script opera de forma independente ou integrado a outros módulos (ex: `app.py` para exibição web).

A classe principal, `DashboardEstatisticas`, encapsula todas as lógicas, utilizando Pandas para manipulação de dados, SQLAlchemy para conexões ao banco e base64 para exportação de gráficos. Ele enfatiza em relatórios robustos, com tratamento de erros e estruturas vazias para falhas, garantindo usabilidade em cenários de dados insuficientes.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via SQLAlchemy), com engine para queries e conexões testadas.
- **Bibliotecas:** Pandas e NumPy (manipulação de dados), Matplotlib e Seaborn (gráficos não-interativos com backend 'Agg'), base64 e io (conversão de figuras), logging (logs de info/erros), datetime (manipulação de datas).
- **Integrações Externas:** Banco de dados 'repo' (tabelas credenciais e credenciais_geral), utils.py (save_plot_to_base64 para gráficos).
- **Scripts Relacionados:** Integra com app.py (exibição de dashboards), password_analyzer.py (análises complementares), credenciais_processor.py (dados processados para stats).

2. Funcionalidades Principais

O `statistics_dashboard.py` fornece ferramentas para geração de estatísticas e visualizações, com foco em métricas agregadas e gráficos. Abaixo, as principais funcionalidades:

- **Consultas ao Banco:** Obtém totais de credenciais, contagens únicas por coluna, itens comuns (senhas, sites, usuários) com limites configuráveis.
- **Cálculos Agregados:** Distribuição mensal de inserções via Pandas (groupby por mês).
- **Geração de Gráficos:** Produz gráficos estáticos em base64 (barplot para top sites, pie para proporção únicos/duplicados) usando Seaborn para estilização.
- **Relatórios Completos:** Compila stats em dict (totais, únicos, comuns, distribuições, gráficos, metadados como timestamp/user).
- **Tratamento de Erros:** Conexões lazy (apenas quando necessário), estruturas vazias em falhas, logs para queries/erros, fechamento automático de conexões e figuras.
- **Visualizações:** Gráficos com temas padronizados (whitegrid, palette 'deep'), grids e labels configurados via rcParams.
- **Integração com Banco:** Queries SQL parametrizadas para segurança, execução com SQLAlchemy para handling de exceções.

3. Configurações

O script define configurações globais e de ambiente via imports de config.py e variáveis na classe:

- **Temporais e Usuário:** self.current_time e self.current_user de CURRENT_TIME/USER (parseados para datetime).
- **Estilos Gráficos:** sns.set_theme (whitegrid, palette='deep'), plt.rcParams (dpi, fonts, axes sizes, grid alpha=0.3, autolayout).
- **Banco de Dados:** Usa get_sqlalchemy_url('repo') para db_uri; SQLALCHEMY_CONFIG para engine (pool, etc.); teste de conexão com "SELECT 1".
- **Logger:** Configurado para 'dashboard_estatisticas' via logging.getLogger.
- **Outras:** Backend Matplotlib 'Agg' (não-interativo); limites default em queries (ex: 10 para itens comuns).

Restrições: Sem schema explícito em queries (assume default); estilos gráficos fixos (edite rcParams para customização); conexões não persistentes (dispose após uso).

4. Funções e Métodos Principais

O script define uma classe `DashboardEstatisticas` com métodos modulares. Cada método é descrito abaixo, incluindo parâmetros, funcionalidades e restrições:

- **init(db_config=None)**: Inicializa engine como None, parseia time/user, configura estilos gráficos.
- **_conectar_db()**: Cria engine lazy se ausente, testa conexão. Raise em SQLAlchemyError ou exceções gerais.
- **_desconectar_db()**: Dispose engine se existente, loga fechamento.
- **_executar_query(query, params=None)**: Executa query parametrizada, retorna lista de dicts. Loga erros, retorna [] em falhas.
- **get_total_credenciais(table_name='credenciais')**: Conta total em tabela. Retorna int (0 se vazio).
- **get_total_unicos(column_name, table_name='credenciais_geral')**: Conta DISTINCT em coluna. Retorna int (0 se vazio).
- **get_senhas_comuns(limit=10)**: Agrupa senhas comuns (WHERE não nula/vazia), ordena DESC, limita. Retorna lista de dicts {senha, quantidade}.
- **get_sites_vazamentos(limit=10)**: Agrupa sites com vazamentos (WHERE não nulo/vazio), ordena DESC, limita. Retorna lista de dicts {dominio, total_vazamentos}.
- **get_usuarios_vazamentos(limit=10)**: Similar para usuários. Retorna lista de dicts {usuario, total_vazamentos}.
- **_calcular_distribuicao_mensal()**: Query data_insercao, usa Pandas para groupby '%Y-%m', retorna dict mes:count. {} em vazio/erros.
- **_get_empty_statistics()**: Estrutura dict vazia para falhas, com chaves para totais, listas, distribuições, gráficos vazios e metadados.
- **gerar_estatisticas_completas()**: Método principal: conecta, calcula totais/únicos/comuns/distribuições, gera gráficos, retorna dict completo. Finally: desconecta e fecha plots. Loga início/sucesso/erros.
- **_gerar_graficos(total_unico, total_geral)**: Gera barplot top sites (viridis palette) e pie únicos/duplicados. Text 'Dados insuficientes' se vazio. Retorna dict de base64.

5. Restrições e Segurança

- **Dados de Entrada:** Assume dados no banco (senhas/sites não nulos); queries sem filtros avançados (ex: datas). Retorna vazios em datasets insuficientes.
- **Gráficos:** Não-interativos ('Agg'); tamanhos fixos (12x7/8x8). Restrição: Fecha todos plots no finally; sem suporte a interatividade.
- **Performance:** Queries agregadas eficientes, mas para grandes tabelas (>1M rows), Pandas pode consumir memória (groupby em RAM). Ajuste pool em config.
- **Segurança:** Queries parametrizadas (contra SQL injection); logs sem dados sensíveis (ex: URIs mascaradas). Raise em erros para propagação.
- **Erros:** Tratados com logger e raise/[]/vazio; conexões com try/except amplo.
- **Restrições Gerais:** Limites hardcoded (10 itens comuns); foco em tabelas específicas (credenciais/geral). Sem exportação direta (integre com app.py para CSV).

Nota Final

O `statistics_dashboard.py` é otimizado para dashboards visuais, mas alterações devem preservar estilos e handling de erros para consistência. Consulte o código para extensões ou adicione gráficos (ex: evolução temporal). Em produção, monitore logs para erros de conexão e atualize bibliotecas para compatibilidade gráfica.